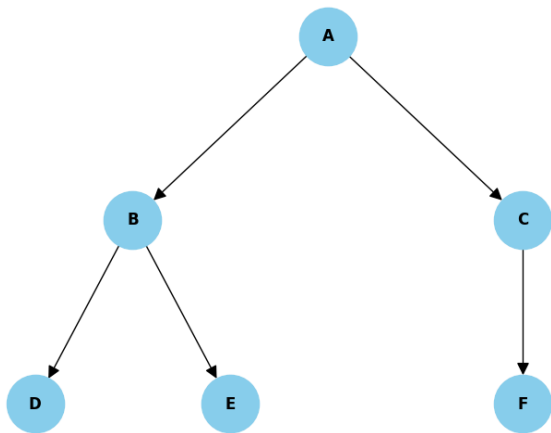


Lab 4

Write a basic program to implement Greedy Best First Search and find the path for the given problem

Example problem



Nodes	Heuristic
A (initial state)	5
B	2
C	3
D	4
E	1
F (goal state)	0

Here, A is the initial state and F is the goal state

Pseudocode

BEGIN

DEFINE adj_list as:

'A' → ['B', 'C']

'B' → ['D', 'E']

'C' → ['F']

'D' → []

'E' → []

'F' → []

DEFINE heuristic values:

'A' → 5

'B' → 2

'C' → 3

'D' → 4

'E' → 1

'F' → 0

FUNCTION gbfs(start, goal):

INITIALIZE visited as empty set

INITIALIZE priority_queue as empty min-heap

INSERT (heuristic[start], start) into priority_queue

INITIALIZE path as empty list

WHILE priority_queue is not empty:

POP (heuristic_value, current_node) from priority_queue

IF current_node IN visited:

CONTINUE

APPEND current_node to path

ADD current_node to visited

IF current_node == goal:

BREAK

FOR each neighbor IN adj_list[current_node]:

IF neighbor NOT IN visited:

INSERT (heuristic[neighbor], neighbor) into priority_queue

RETURN path

FUNCTION display_adjacency_list(adj_list):

PRINT "Adjacency List:"

FOR each node, neighbors IN adj_list:

PRINT node → neighbors

FUNCTION main:

CALL display_adjacency_list(adj_list)

INPUT start node

INPUT destination node

CONVERT both to uppercase

CALL gbfs(start, destination)

STORE result in path

PRINT "Path from [start] to [destination]:"

FOR each node IN path:

PRINT node →

PRINT "End"

CALL main IF this is the main program

END

Report format

Title: Write a program

Date: Lab assigned date

Objective

- To implement the Greedy Best First Search algorithm using an adjacency list to explore nodes in a graph or tree structure.
- To find and display the path from a given start node to a destination node, if such a path exists, using Greedy Best First Search traversal.

Theory

- Greedy Best First Search Algorithm

Implementation

- Write down the problem statement
- Python Code
- Output (Screenshots of your output)

Conclusion